Huskysort

R C HILLYARD*, College of Engineering, Northeastern University, USA
YUNLU LIAOZHENG†, College of Engineering, Northeastern University, USA
SAI VINEETH K R‡, College of Engineering, Northeastern University, USA

Much of the copious literature on the subject of sorting has concentrated on minimizing the number of comparisons and/or exchanges/copies. However, a more appropriate yardstick for the performance of sorting algorithms is based on the total number of array accesses that are required (the "work"). For a sort that is based on divide-and-conquer (including iterative variations on that theme), we can divide the work into linear, i.e. $\mathbf{O}(N)$, work and linearithmic, i.e. $\mathbf{O}(NlogN)$, work. An algorithm that moves work from the linearithmic phase to the linear phase may be able to reduce the total number of array accesses and, indirectly, processing time. This paper describes an approach to sorting which reduces the number of *expensive* comparisons in the linearithmic phase as much as possible by substituting inexpensive comparisons. In Java, the two system sorts are dual-pivot quicksort (for primitives) and Timsort for objects. We demonstrate that a combination of these two algorithms can run significantly faster than either algorithm alone for the types of objects which are expensive to compare. We call this improved sorting algorithm Huskysort.

CCS Concepts: • **Theory of computation** → **Sorting and searching**; **Data structures and algorithms for data management**; • **Software and its engineering** → **Data types and structures**; • **Computer systems organization** → Multicore architectures;

Additional Key Words and Phrases: Sorting, Quicksort, Timsort, Insertion sort, Complexity, Comparison Sorting

ACM Reference Format:
R C HILLYARD, Yunlu Liaozheng, and Sai Vineeth K R. 2020. Huskysort. *ACM Trans. Graph.* 37, 4, Article 111 (August 2020), 13 pages. https://doi.org/10.1145/1122445.1122456

1 INTRODUCTION

With the vast increase in the size of the data world, efficient computing algorithms play a crucial role in processing data. From the earliest days of computing, sorting has been one of the most studied areas of research even though it appears to be a straightforward and familiar topic. The rise of data volume and complexity has made clear a need for new approaches to traditional sorting techniques. Within the realm of comparison-sort algorithms, many aspects of the algorithm can be significant: stability, extra memory, partitioning, merging, exchanging (swapping), copying, and, of course, the comparison itself.

Counting memory operations rather than only comparisons is itself an established idea, and we situate our own array-access framing relative to two well-known lines of work in that tradition rather than proposing it as new: the external-memory (I/O) model of Aggarwal and Vitter [Aggarwal and Vitter 1988], which counts block transfers between disk and memory, and the cache-oblivious algorithms of Frigo et al. [Frigo et al. 1999], which achieve near-optimal memory-hierarchy behaviour without knowing cache parameters in advance. Our array-access count is a simpler, single-level analogue of the same underlying idea: rather than modeling an explicit multi-level memory hierarchy, we count raw array reads and writes as a proxy for real cost, motivated by the observation that a comparison between two heavyweight objects (e.g. two *String*s) touches far more memory than a comparison between two 64-bit primitives. That observation is orthogonal to, not a substitute for, the external-memory and cache-oblivious literature, which is chiefly concerned with data too large to fit in memory (or in cache) at all, rather than with the comparison cost of individual elements that do fit. Separately, radix sort's independent per-digit counting-sort passes are well suited to parallelization across CPU threads, a property that is well established for radix sort generally; § 6.4 describes and benchmarks a parallel variant of RadixHuskySort (§ 3.2). We do not make an equivalent claim for Huskysort's own comparison-based steps: Introsort's partitioning and Timsort's adaptive run-detection are both substantially more data-dependent and sequential in character than radix sort's fixed per-digit passes, so parallelizing them well is a separate, harder problem that this paper does not address.

Even though sorting appears to be a settled area of research, yet new improvements are still being made: [Bentley and McIlroy 1993; Jugé [n.d.]; McIlroy 1993; Peters 2002; Yaroslavskiy 2009]. There is, therefore, an abundance of sorting techniques available. Nevertheless, there is still room for improved versions in regard to computation, memory, time, and space complexity. In a recent paper [Zhang et al. 2016], the authors conclude that quicksort is indeed the fastest general-purpose sorting algorithm, especially when appropriate care is taken to avoid the quadratic worst-case with several other improvements (e.g. dual-pivot quicksort)[Yaroslavskiy 2009] over the traditional version [Hoare 1961]. However, quicksort does not perform as well as some variations of merge sort, especially Timsort [Peters 2002], when the dataset is partially sorted. Timsort, a hybrid stable sorting algorithm derived from merge sort, is now available in the Python and Java libraries as the default object-sorting utility.

*originator of the idea; did the analysis; designed, coded, and performed much of the benchmarking. The developed Huskysort Code is available at: https://github.com/rchillyard/HuskySort
†implemented the initial algorithm, including the coders and the key innovation of the object swap in the quicksort phase; early benchmarking.
‡Analysed the data, extended the literature survey and wrote the paper with input from all authors.

Authors' addresses: R C HILLYARD, r.hillyard@northeastern.edu, College of Engineering, Northeastern University, 360 Huntington Avenue, Boston, Massachusetts, USA, 02115; Yunlu Liaozheng, liaozheng.y@northeastern.edu, College of Engineering, Northeastern University, 360 Huntington Avenue, Boston, Massachusetts, USA, 02115; Sai Vineeth K R, kandappareddigari.s@northeastern.edu, College of Engineering, Northeastern University, 360 Huntington Avenue, Boston, Massachusetts, USA, 02115.

0730-0301/2020/8-ART111 $15.00
https://doi.org/10.1145/1122445.1122456

We determined to test the efficacy of combining quicksort with Timsort for object types that are costly to compare. Such types include *String, LocalDateTime, BigInteger, BigDecimal*, and many user-defined types of this kind.

We review existing methods of sorting in Section 2 and go through the Huskysort algorithm in Section 3. Implementation options and their likely impact are explored in Section 4. We describe our Test Cases in Section 5 and present results in Section 6, with outcomes summarized in Section 7.

2 BACKGROUND

Before introducing Huskysort, it is worth laying out the four comparison-based algorithms it draws on side by side, since each contributes a different property to the final design. Table 1 summarizes their standard complexity and stability characteristics.

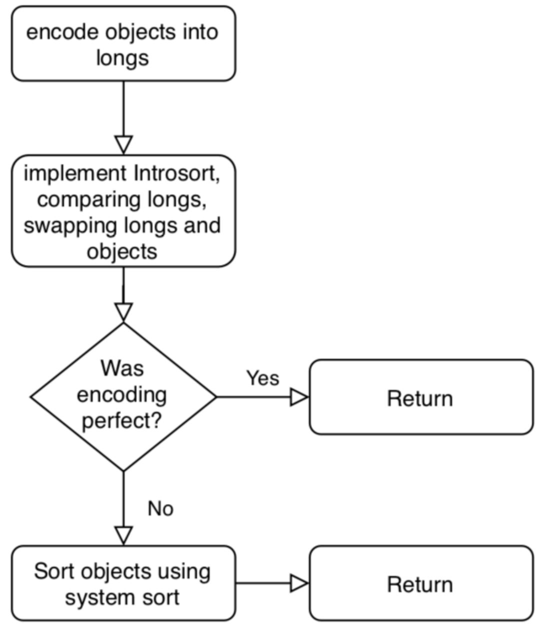
Quicksort is fastest in practice on unordered data (its actual average case), because its in-place partitioning and cache-friendly access pattern outweigh merge sort's guaranteed $\mathbf{O}(N \log N)$ bound; but it buys that speed at the cost of stability, and its worst case is quadratic unless mitigated (as Introsort does, by falling back to heapsort past a recursion-depth threshold — see § 3.2 below). Timsort takes the opposite trade: guaranteed $\mathbf{O}(N \log N)$ worst case and stability, at the cost of being merge-sort-based (extra space, and no faster than $\mathbf{O}(N \log N)$ on unordered data) — but it is adaptive, so on data that is already close to sorted (few inversions, I), its cost drops toward $\mathbf{O}(N)$, which quicksort's partitioning does not exploit in the same way. Huskysort's own strategy follows directly from this contrast: husky-coding turns step 2's input into something quicksort (or, as described in § 3.2, radix sort) can sort at its best case — the coded longs are effectively unordered, cheap-to-compare data — and reserves Timsort for exactly the regime where it is strongest: cleaning up the (hopefully few) remaining inversions in step 3, where the array is already close to sorted rather than random.

Timsort is ideally suited to the partially ordered data which is common in the real-world business applications. For this use case it is easily the fastest standard algorithm. It is less well suited, however, for another use case, which is unordered data. Huskysort is designed specifically for this use case: unordered data consisting of objects which are relatively expensive to compare.

Before talking about our approach let's look at insertion sort and Timsort in more detail. The key feature of both of these methods is that the number of comparisons (and swaps, in the case of insertion sort) is proportional to the number of inversions. Because insertion sort is an "elementary" sort (it reduces the complexity of the problem by iteratively partitioning by one element at a time), it is only suitable for small arrays. It traverses the array, moving each new element encountered into its proper place in the ordered partition, which therefore grows by one element in each iteration until the whole array is ordered.

Timsort is a variation of bottom-up merge sort which takes advantage of existing ordered runs. Because of this, it is very efficient for partially-ordered arrays. Additionally, like merge sort, Timsort is stable. Timsort's worst case is as $\mathbf{O}(N + N logp)$, where p is the number of runs [Auger et al. 2018].

Fig. 1. Huskysort Flow



One of the authors (Hillyard) had set a question in a mid-term exam (Fall, 2018) about a hypothetical algorithm that created a 64-bit hash code for a date-time object and then used quicksort to sort the array. Afterward, it became apparent that it was not such a crazy idea after all. A quick implementation showed that, indeed, we could see an improvement over the system sort (Timsort) for sorting objects. The essence of the algorithm lies in the fact that quicksort can sort an array of (64-bit) longs very quickly. 64 bits affords sufficient room to encode many objects uniquely, but even if the coding is not unique, post-sorting an array of objects which are quasi-ordered can be very fast using Timsort. The key innovation of the algorithm is to swap the objects at the same time as swapping the longs.

3 ALGORITHM

3.1 Overview

The strategy of Huskysort (see Flow 1) is as follows (assuming that the dataset is presented as an array xs of type X): see Algorithm 3

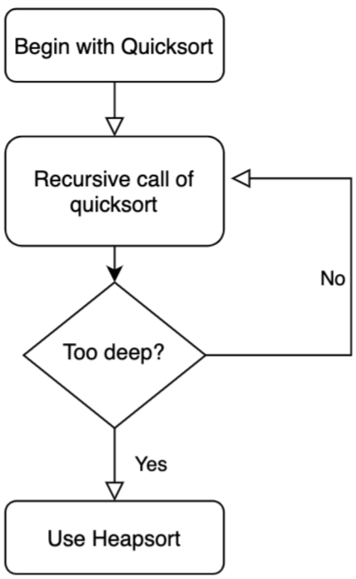
Step 1: a Create a $long[]$ array (*longs*) of the same size as xs;
b for each element x of xs, set the corresponding element of *longs* to $x.huskyCode()$.

Step 2: Sort array *longs* using quicksort (in practice, we have found that Introsort [MUSSER 1997] works well) in such a way that

Table 1. Prior comparison-based sorting algorithms

Algorithm	Average case	Worst case	Stable	Notes
Insertion sort	$\mathbf{O}(N+I)$	$\mathbf{O}(N^2)$	Yes	I = number of inversions; fast only for small or nearly-sorted N
Merge sort	$\mathbf{O}(N \log N)$	$\mathbf{O}(N \log N)$	Yes	Linearithmic regardless of input order; $\mathbf{O}(N)$ extra space
Quicksort	$\mathbf{O}(N \log N)$	$\mathbf{O}(N^2)$	No	In-place, cache-friendly; the quadratic worst case is mitigated in practice (e.g. Introso
Timsort	$\mathbf{O}(N+I)$ to $\mathbf{O}(N \log N)$	$\mathbf{O}(N \log N)$	Yes	Hybrid of merge sort and insertion sort, adaptive to existing ascending/descending ru

Fig. 2. Introsort Flow



when elements i, j of *longs* are exchanged, then elements i, j of xs are also exchanged; See Flow 2

Step 3: Finally, when quicksort is finished and, if necessary, we run Timsort on the array xs to ensure that its elements are truly in order.

For the details of the husky-coding itself, see Algorithm 6

3.2 RadixHuskySort

Step 2 sorts the husky codes using Introsort, an $\mathbf{O}(NlogN)$ algorithm, but the codes themselves are fixed-width 64-bit values, and radix sort can sort k-bit keys in $\mathbf{O}(N \cdot \frac{k}{b})$ time using b-bit digits — linear in N for any fixed digit width b. Given that Huskysort's entire premise is to move work from the linearithmic phase to the linear phase, this raises a natural question we had not previously addressed: could step 2 itself be made linear? This section describes RadixHuskySort, which we built to answer it (Algorithm 1).

A naive adaptation — swapping the object reference alongside each long at every exchange within every digit pass, exactly as step 2's Introsort swaps at every partition exchange — would incur $\mathbf{O}(N)$ reference movements per digit pass, or $\mathbf{O}(N \cdot \frac{64}{b})$ movements in total. Since $\frac{64}{b}$ is typically much smaller than $logN$ at the sizes we tested, this would still usually beat Introsort's swap cost, but it needlessly reintroduces exactly the kind of payload-movement overhead this whole design exists to avoid, and least-significant-digit (LSD) radix sort's per-pass counting-sort structure makes a better option available.

Instead, RadixHuskySort's step 2 carries only an *int[]* index array through the LSD digit passes, alongside the (unmodified) long array; each pass is a stable counting sort keyed on one b-bit digit of the (already-computed) husky code, reordering the index array rather than the codes or the objects themselves. Once all $\frac{64}{b}$ passes are complete, the index array holds the sorting permutation, and a single $\mathbf{O}(N)$ pass builds the final object array by reading *xs[index[i]]* for each i. This confines all object-reference movement to one pass total, rather than one pass per digit.

Because LSD radix sort's per-digit counting sort treats keys as unsigned, and husky codes are signed 64-bit values whenever the underlying domain can be negative (e.g. *Integer*, *Long*, or dates before a chosen epoch), each code is first transformed by $code \oplus$ *Long.MIN_VALUE* — flipping only the sign bit — before the digit passes, which places negative codes before non-negative ones under unsigned digit comparison without disturbing relative order within either group; the same transform is idempotent to reverse, so no separate un-flip step is needed once the permutation is built from the transformed codes. Whether a given *HuskyCoder* can actually produce negative codes is checked rather than assumed, since several codings (e.g. Unicode strings) never do.

The digit width b is a tunable parameter, trading off the number of passes ($\frac{64}{b}$) against the size of the per-pass counting array (2^b buckets, which must fit comfortably in cache for each pass to stay fast); § 6.3 reports how this trade-off plays out empirically across data types and array sizes. Step 3 (the cleanup pass, run only if the coding is not provably *perfect*) is unchanged regardless of which algorithm sorts the codes in step 2 — radix sort is a drop-in replacement for that one step, not a change to the overall three-step strategy.

This parameterization also answers a separate concern raised about the original design: that it assumes 64-bit words throughout, which may not be appropriate for comparison on different key sizes. Quicksort's asymptotic behaviour does not depend on key width at all — an $\mathbf{O}(N \log N)$ comparison sort costs the same whether the keys being compared are 32 bits or 128 bits wide, since only the (fixed-cost) comparison itself touches the key's actual bits. Radix sort, by contrast, makes the key width an explicit, first-class parameter: a k-bit key simply takes $\frac{k}{b}$ passes instead of $\frac{64}{b}$, with no change to the algorithm itself. So while nothing about the discussion above depends on 64-bit words specifically, the RadixHuskySort generalizes to other key widths more directly than a comparison-based sort does — if a future encoding needed, say, 128 bits to capture more of an object's structure, the same radix sort would apply unchanged, merely with twice as many passes.

Radix sort applied to strings specifically has its own substantial, and older, literature, distinct from the general fixed-width-key radix sort described above. Three-way radix quicksort (also called multikey quicksort) and MSD (most-significant-digit) radix sort for strings were both popularized by Bentley and Sedgewick's classic treatment [Bentley and Sedgewick 1997], and burstsort [Sinha and Zobel 2004] improves cache behaviour further by building a small trie to bucket strings before sorting. These string-specialized algorithms sort variable-length strings directly, character by character, without a separate fixed-width encoding step. RadixHuskySort's contribution is different in kind, not degree: it is not a string-sorting algorithm, but a general mechanism applicable to any *Comparable* type with a 64-bit husky encoding (strings among them), at the cost of the encoding's own fixed capture window (§ 3.2 above) and a fallback cleanup pass whenever that encoding is imperfect. A direct empirical comparison against these string-specialized algorithms, for the specific case of sorting only strings, would be a natural next step, but is outside the scope of this paper.

Algorithm 1: RadixHuskySort.sort

Input: *xs as an array of object X which extends Comparable<X>*
Input: *b as the number of bits per digit*
coding = huskyCoder.huskyEncode(xs);
longs = coding.longs;
index = radixSortIndices(longs, b);
ys = permute(xs, index);
if *coding.perfect* **then**
    return ys;
Arrays.sort(ys);

Algorithm 2: radixSortIndices (LSD, deferred permutation)

Input: *longs as an array of long*
Input: *b as the number of bits per digit*
Output: *index, a permutation of* $0, \ldots, longs.length - 1$ *such that longs[index[·]] is sorted*
biased[i] = longs[i] **XOR** Long.MIN_VALUE, for each i;
index = [0, 1, …, longs.length - 1];
for *digit = 0* **to** *(64 / b) - 1* **do**
    index = stableCountingSortPass(biased, index, digit, b);

3.3 Discussion of Husky Encoding

Step 1b requires the definition of a 64-bit hash function (huskyCode) whose properties are as follows: (refer to Algorithm 4)

- If $x = y$, then $x.huskyCode() = y.huskyCode()$;
- If $x > y$, then:
  - either $x.huskyCode() <= y.huskyCode()$ (with probability p)
  - or $x.huskyCode() > y.huskyCode()$ (with probability $1 - p$);

We interpret this rule as follows: if x and y are the same ($x.equals(y)$ in Java), their huskyCode should be the same. If $x > y$, we normally expect that $x.huskyCode() > y.huskyCode()$, i.e. the sort on the husky-coded longs will not invert x and y. However, when $x > y$ while $x.huskyCode() < y.huskyCode()$, then an inversion will be introduced. And, when $x > y$ and $x.huskyCode() = y.huskyCode()$, then an inversion *may* be introduced because quick-sort is not stable.[Xiang 2011]

For a successful encoding, the value of p, the probability of a potential inversion, should be less than some critical probability, p_{crit}. Leaving a more detailed discussion of p_{crit} until later, we note that, for some domains of X, we can definitely assert that $p = 0$. We refer to this as a "perfect" encoding. In such a case, we can skip step 3 of the algorithm because, when $p = 0$, there will be no inversions remaining after quick-sorting according to the huskyCodes. An example of such a domain is date-time values with a resolution of one nano-second over a period of 128 years. For strings of English case-independent letters (no punctuation or numbers), i.e. 5-bit ASCII, words of 12 (or fewer) characters in length can also be coded uniquely. For case-dependent strings (6-bit ASCII), we can encode perfectly if the lengths are all 10 characters or less. In step 3, it might be acceptable to use insertion sort instead of Timsort. In any event, assuming that the huskyCode is as accurate as required, then there will be very few inversions (elements out of place) remaining after the quicksort phase. This implies that the second sort can be performed in very close to linear time. The results have been very satisfactory. For example, taking an array of 100,000 English words chosen randomly from a corpus of 100,000 words [Goldhahn et al. 2012], we have found, for example in one run, that Huskysort takes a normalized time of 0.93, compared with 1.00 for quicksort and 1.45 for Timsort (For further data, please refer to table 6.)

3.4 Discussion of p_{crit}

When $p > 0$, there is a finite probability of any pair of elements remaining inverted after the first sorting pass. For an array of N randomly ordered elements, the average number of inversions before sorting is $X = N(N-1)/4$. The number of inversions remaining after the first pass is simply pX. And, for Timsort or insertion sort, the time to re-sort the array will be $t = k(N + pX)$ where k is some system-dependent constant. The total time to sort the array is made up of these three time-consuming steps where k_1, k_2, k_3 are system- and algorithm-dependent constants:

Step 1: T_1 is the time to encode the elements:

$$T_1 = k_1 N$$

Step 2: T_2 is the (average) time to quick-sort the huskyCode array while also swapping the element array itself:

$$T_2 = 2k_2 N ln N$$

Step 3: T_3 is the time to Timsort the element array (only required if $p > 0$):

$$T_3 = k_3(N + pX)$$

Provided that the total time $T = T_1 + T_2 + T_3$ is less than the time required to use Timsort on the original element array, we have a win. This same three-step decomposition ($T_1/T_2/T_3$, i.e. encode/-sort/cleanup) is revisited in § 5.2 in more concrete terms — counting array accesses directly rather than folding everything into abstract system-dependent constants k_1, k_2, k_3 — and extended there to cover RadixHuskySort (§ 3.2) as a fourth alternative for the T_2 step. The actual values of p_{crit} are very much dependent on the machine running the sort. We have not calculated a typical value for p_{crit} but, even for the worst-case: Chinese characters using UTF8 encoding, we still find very significant gains, summarized later in Table 11 (§ 6.2).

3.5 Why Huskysort Works

So, why does this work? Well, first of all, we observe that 64-bit word length in today's computers is the norm. This allows for an encoding which has a low, possibly zero, value of p_{crit}.

Even though a Unicode string can encode only the first 3 15/16ths characters in 64 bits, this is extremely helpful in getting the elements close to their final order. The 15/16ths comes from a final unsigned right shift by one bit, applied after packing four 16-bit characters into the 64-bit code: without it, any string beginning with a character whose code point is 0x8000 or above (common among CJK characters) would set the sign bit and be encoded as a negative *long*, corrupting simple numeric comparison. The shift costs exactly the least-significant bit of a true fourth character; for strings of three characters or fewer, that bit is always an unused padding zero, so nothing is actually lost — which is exactly why this encoding is only declared *perfect* up to three characters, not four.

Here, we may compare the method of Huskysort with that of ShellSort.[Sedgewick 1996]

ShellSort acts exactly like a series of insertion sorts but, whereas each compare/swap in insertion sort is limited to resolving one inversion only, so in ShellSort, h inversions can be resolved in one compare/swap (where we are h-sorting the array). So, Shellsort's primary mechanism is to try to fix as many inversions as possible before taking the subsequent step.

In the same way, Huskysort fixes as many inversions as possible before doing the final step. Note that it isn't necessary for the encoding to be perfect. But it should be good.

Another way of looking at it is that running quicksort or merge sort (or their variations) will require $\mathbf{O}(NlogN)$ comparisons. Merge sort, like insertion sort, is linear when the number of inversions is small and grows with $\mathbf{O}(N)$. But merge sort does use extra memory. But values such as 64-bit integers (int or long), doubles, can be compared extremely quickly in modern machines—far quicker than the time required to follow a reference (pointer) to an object in memory. In Java, this distinction is described by contrasting primitives with objects. Primitives can be quick-sorted very fast, and *long* is a primitive data type.

Exchanging (swapping), on the other hand, takes the same amount of time whatever type you are sorting because swapping an object reference is just the same as swapping any 64-bit word. The other reason why quicksort is so fast is that it is cache-friendly: its in-place partitioning scans memory sequentially rather than jumping between widely-separated locations, giving it markedly better cache behavior than, for example, heapsort [LaMarca and Ladner 1999]. Cache behavior also distinguishes quicksort variants from one another, not just quicksort from other algorithm families: dual-pivot quicksort's own performance advantage over classic single-pivot quicksort has likewise been attributed largely to cache effects [Yaroslavskiy 2009]. (LaMarca and Ladner's study also found that a naive radix sort, sorting keys directly rather than through a deferred index permutation, has relatively poor cache behavior of its own on 1990s hardware — a finding about a different radix sort design than § 3.2's, on much older hardware, so it does not bear on this paper's own radix-sort results.) So, we perform a fast quicksort on the array of longs. The only difference between the Husky version of quicksort and standard quicksort is that the swap operation must actually do two swaps: one of the longs and one of the object references. But, again, swapping is fast because elements of both arrays (i.e. both longs and refs) have good chances of being cached.

The extra time (and space) required to perform the encoding upfront, which is $\mathbf{O}$(N), is more than offset by the faster comparisons of longs as opposed to comparisons of objects.

After the first pass (using quicksort), we can easily determine whether our encoding was perfect—in which case, there will be no remaining inversions. We do this for non-sequence classes via a simple constant-time lookup. For sequences (in practice, these are always Strings), we check each sequence's coding perfection (according to its length), and as soon as we find a sequence that cannot be encoding perfectly, we stop checking.

If the coding was perfect, we simply skip the final pass.

Given the notion of hashing that the algorithm uses, it would seem natural to call it Hash Sort. Unfortunately, that name was already taken for quite a different algorithm [Gilreath 2004]. We chose the name Husky Sort instead, after the husky, Northeastern University's mascot, where this work was developed.

Algorithm 3: Huskysort.sort

Input: *xs as an array of object X which extends Comparable<X>*

```
coding = huskyCoder.huskyEncode(xs);
longs = coding.longs;
Introsort(xs, longs, 0, longs.length, 2 * floor_lg(xs.length));
if coding.perfect then
    return;
Arrays.sort(xs);
```

Algorithm 4: huskyEncode

Input: *xs as an array of object X*
Output: A new *Coding object*
result = new long[xs.length];
for *i = 0 to xs.length* **do**
    result[i] = huskyEncode(xs[i]);
return new Coding(result, perfect());

Algorithm 5: huskyEncode (for character sequence)

Input: *xs as an array of CharSequence X*
Output: A new *Coding object*
isPerfect = true;
result = new long[xs.length];
for *i = 0 to xs.length* **do**
    X x = xs[i];
    if *isPerfect* **then**
        isPerfect = perfectForLength(x.length());
    result[i] = huskyEncode(xs[i]);
return new Coding(result, isPerfect);

Algorithm 6: stringToLong

Input:
str as String;
maxLength as the max length can be encoded;
bitWidth as how many bits will one char be encoded into;
mask as a modifier for different encoding;
Output: A new *Coding object*
length = Math.min(str.length(), maxLength);
padding = maxLength - length;
result = 0;
if *mask == 0* **then**
    for *i = 0 to length* **do**
        result = result « bitWidth | str.charAt(i);
else
    for *i = 0 to length* **do**
        result = result « bitWidth | str.charAt(i) & mask;
result = result « bitWidth * padding;
return result;

4 IMPLEMENTATION

4.1 System Environment

Table 2 lists the configuration used for the original benchmarks (the JDK 1.8.0_152-based results reported in § 5.2 and Table 6). A natural question is whether these results generalize to other machines and JVM versions, since only one environment was used there. Table 3 lists the very different environment used for the radix-sort and JMH work of § 3.2-§ 6.3, run several years later on different hardware and a current JVM. The qualitative finding (Huskysort, and now radix sort, beating System sort) held up unchanged across this substantial a difference in both machine and JVM version.

Table 2. Original benchmark environment (2020)

Model	MacBook Pro (MacBookPro14,3)
Processor	Quad-Core Intel Core i7, 2.8 GHz, 1 processor, 4 cores, Hyper-Thre
Cache	256 KB L2 (per core), 6 MB L3
Memory	16 GB 2133 MHz LPDDR3
JVM	1.8.0_152

Table 3. Current benchmark environment (2026, radix sort and JMH work)

Model	MacBook (Apple Silicon)
Processor	Apple M1, 8 cores (4 performance + 4 efficiency)
Cache	128 KB L1I / 64 KB L1D per core; 12 MB L2 (performance cluster), 4
Memory	16 GB unified
OS	macOS 26.5.2
JVM	OpenJDK 21.0.10

Table 4. String-encoding constants

Constant	Unicode	ASCII
Bit width per character	16	7
Max length (64 / bit width)	4	9
Mask	`0xFFFF`	`0x7F`

4.2 Implementation of Algorithm

The top-level *sort* method and the default *huskyEncode* method are implemented directly as shown in Algorithms 3 and 4 above; rather than repeat that logic here as Java source, we describe only the details not already covered by those two listings.

Coding is a simple immutable holder combining the resulting *long[]* array of codes with the *perfect* flag used by both algorithms (two fields, set once by its constructor).

unicodeToLong and *asciiToLong* are both thin wrappers around *stringToLong* (Algorithm 6), differing only in which bit-width, maximum length, and mask constant they pass in; *unicodeToLong* additionally discards the lowest bit of the packed result (an unsigned right shift by one) to guarantee a non-negative code regardless of the input string's content. Table 4 lists the constants each wrapper uses.

5 TEST CASE AND ANALYSIS

5.1 Data Source

The sources for the Strings (English and Chinese) are retrieved from Leipzig Corpora Collection [Goldhahn et al. 2012].

In each case, we take data from the appropriate *sentences.txt* file and break it up into strings as shown in Listing 1.

The sample of data can be seen here 3

```
private static String[] getLeipzigWordsFromResource(final
      String resource) {
    return getWords(resource, HuskySortBenchmark::
     getLeipzigWords);
}
```

Fig. 3. Strings Data Example

Example of English strings:	Example of Chinese strings:
words = {String[22865]@1158}	words = {String[24017]@1388}
0 = "Consider"	0 = "Aaron"
1 = "the"	1 = "如果您指的是"
2 = "extras"	2 = "过滤效果"
3 = "not"	3 = "Bronkhorst"
4 = "usually"	4 = "Accept"
5 = "included"	5 = "Ranges"
6 = "with"	6 = "bytes"
7 = "any"	7 = "Last"
8 = "FREE"	8 = "Modified"
9 = "car"	9 = "Fri"
10 = "offer"	10 = "Mon"
11 = "CDW"	11 = "Sat"
12 = "LDW"	12 = "Acid"
13 = "For"	13 = "Alkaline"
14 = "noise"	14 = "Exhaust"
15 = "which"	15 = "Fan"
16 = "is"	16 = "System"
17 = "persistent"	17 = "Solvent"
18 = "but"	18 = "Treatment"
19 = "occurs"	19 = "General"
20 = "on"	20 = "一般气体排风机系统"
21 = "such"	21 = "制程中产生之一般无害气体"
22 = "an"	22 = "DMFu是dimethyl"
23 = "irregular"	23 = "fumarate"
24 = "basis"	24 = "也就是富马酸二甲酯"
25 = "that"	25 = "Adobe"
26 = "case"	26 = "表示让全校师生都能享有最新的"
27 = "officer"	27 = "Creative"
28 = "unable"	28 = "Suite"
29 = "to"	29 = "软件与技术"

```java
private static List<String> getLeipzigWords(final String line) {
    return HuskySortBenchmarkHelper.splitLineIntoStrings(line, REGEX_LEIPZIG, REGEX_STRINGSPLITTER);
}
final static Pattern REGEX_LEIPZIG = Pattern.compile("[~\\t]*\\t(([\\s\\p{Punct}\\uFF0C]*\\p{L}+)*)");
public static final Pattern REGEX_STRINGSPLITTER = Pattern.compile("[\\s\\p{Punct}\\uFF0C]");


static List<String> splitLineIntoStrings(final String line, final Pattern lineMatcher, final Pattern stringsplitter) {
    final Matcher matcher = lineMatcher.matcher(line);
    if (matcher.find()) return Arrays.asList(stringsplitter.split(matcher.group(1)));
    else return new ArrayList<>();
}

static String[] getWords(final String resource, final Function<String, List<String>> stringListFunction) {
    try {
        File file = new File(getPathname(resource, DutchHuskySort.class));
        return getWordArray(file, stringListFunction, 2);
    } catch (FileNotFoundException e) {
        logger.warn("Cannot find resource: "+resource, e);
        return new String[0];
    }
    } catch (final Exception e) {
        throw new RuntimeException(e);
    }
}
```

Listing 1. Data Source

5.2 Analysis

Let's try to do an analysis of Huskysort's performance. This expands on the $T_1/T_2/T_3$ decomposition introduced in § 3.4 above, replacing its abstract time constants with concrete array-access counts, and extending it to cover radix sort (§ 3.2) as an alternative to quicksort for the T_2 step. The first thing to note is that, like merge sort, Huskysort requires $\mathbf{O}(N)$ extra space. This is because, in addition to the array of object references, we must have an equal-length array of longs (the huskyCode values).

Secondly, Huskysort is not stable unless using the merge sort variation. How many comparisons and swaps does Huskysort require, on average? It has been shown for dual-pivot quicksort [Nebel] that these values are (approximately):

- Comparisons: $1.9 N ln N$;
- Swaps: $0.6 N ln N$.

Our experiments [refer to Quicksort Analysis table] show that each comparison requires 2 array accesses (although, because one comparand is always a pivot value, this number is effectively closer to 1. Each swap requires 4 array accesses, although again, we will be swapping elements that have already been accessed. The effective number of (normalized) array accesses is, in total, approximately 4 rather than the expected 3.8 + 2.4. That is to say, the coefficient of $N ln N$ is approximately 4. The analysis represented by the table is for dual-pivot quicksort. [1] However, in Huskysort, each swap also must swap the object references. These will not, in general, have been pre-fetched and so the effective total normalized array accesses (A) is approximately:

$$A = 4 + 0.6 * 4 = 6.4$$

Additionally, Huskysort requires N huskyCode constructions. Each of these requires $k + 1$ array accesses where k is proportional to the number of fields contributing to the huskyCode. For a string, that will be the number of characters in the string (except that this is limited to a relatively small number according to the encoding). For ASCII strings, it is nine. For Unicode strings, it is four.

However, keep in mind that this is linear and, while it can't be completely ignored, it doesn't contribute to the overall growth of the complexity. This was previously an assumption rather than a measurement; we have since timed the encoding phase in isolation, separately from the sort that follows it, to confirm it directly. At N = 1,000,000, encoding takes 52.7ms for English words (13.7% of QuickHuskySort's 383.2ms total, 21.1% of RadixHuskySort's 249.3ms total) and 311.6ms for Chinese names encoded via pinyin (29.1% of QuickHuskySort's 1070.2ms total, 40.0% of RadixHuskySort's 779.4ms total) — confirming that encoding cost, while real and non-negligible, especially for the more elaborate pinyin encoding, remains a minority of total time in every case measured, consistent

[1]Note that, since dual-pivot quicksort is not actually implemented in the Java library for Comparable objects, we re-implemented the class for objects.

with the linear-and-subordinate role it is assumed to play above. Encoding's share of the total is larger for RadixHuskySort than for QuickHuskySort in both cases, simply because radix sort's own contribution to the total is smaller — as the sort phase gets cheaper, whatever cost remains in the encoding phase necessarily becomes a proportionally larger slice of what is left, which suggests encoding cost (particularly for pinyin) is the more promising target for further optimization now that the sort phase itself is no longer the dominant cost. Once the Huskysort is complete, we will assume that there are pN remaining inversions, where p is the probability of an idiogenic (self-inflicted) inversion. Again, while we should not ignore it completely, it is linear. The number of array access for the comparisons/swaps to fix these will be $(2 + j + 4)pN$, where j is the number of fields to be compared to get a result (j is usually smaller than k).

So, the total number of array accesses (A) for Husky-sorting an array of length N is:

$$A = (6.4lnN + (2 + j + 4)p + k + 1)N$$

Timsort would be hard to analyze in the general case and so we instead analyze merge sort, which should be approximately the same number of array accesses in the average case.

For merge sort, the analysis is much simpler: the number of comparisons for a randomly ordered array of length N is $Nlog_2N$. Merge sort does not use swaps, *per se*, but instead uses copies. Assuming that we optimize the extra copying, there will be N copies to get the algorithm started and N per level. Thus, the number of copies will be $Nlog_2N$. Each of these copies requires two array accesses while each comparison requires 2 + j (described above) array accesses.

The total number of array accesses (A) for merge sort is, therefore:

$$A = (4 + j)Nlog_2N + 2N$$

Given that $log_2N = 1.44lnN$, and assuming that:

$j = 4$ corresponding to the comparison of two strings that start with the same character but differ in the second character; and

$k = 7$ and $p = 0.1$; then

the totals (A_m for merge sort and A_h for Huskysort) come to:

$$A_m = 11.5NlnN + 2N$$

$$A_h = 6.4NlnN + 9N$$

The consequence of this is that, as expected, the linear phase of Huskysort is relatively high for small N but that is soon swamped by the saving of comparisons in the linearithmic phase. The break-even point is actually a very small value of N: four. As N increases, we expect the advantage of Huskysort to increase further. This is generally borne out by the benchmarks. Please refer to tables 5 and 6.

The same style of analysis extends naturally to RadixHuskySort (§ 3.2). Each of its $\frac{64}{b}$ digit passes touches every element a small constant number of times, c, to bucket it and to record its new position in the (deferred) index array — with no partitioning or pivot-comparison overhead, and critically, no logarithmic term at all. Taking $c = 4$ (the same per-exchange cost used for Huskysort's swaps above), adding the final $\mathbf{O}(N)$ pass that applies the resulting permutation to the object array (2 array accesses per element), and the same encoding cost ($(k + 1)N$) and cleanup-pass cost ($(2 + j + 4)pN$, unaffected by which algorithm sorts the codes in step 2), the total array accesses for RadixHuskySort (A_r) come to:

$$A_r = \left(4 \cdot \frac{64}{b} + 2 + k + 1\right) N + (2 + j + 4)pN$$

Using the same illustrative constants as above ($j = 4$, $k = 7$, $p = 0.1$) and $b = 16$ (four digit passes, near the middle of the digit-width plateau found empirically — see § 6.3):

$$A_r = 27N$$

Table 5 extends the earlier comparison with this term. At $N =$ 1,000,000, $A_r \approx 27{,}000{,}000$ against $A_h \approx 101{,}000{,}000$ and $A_m \approx 161{,}000{,}000$ — a predicted advantage of roughly 3.7x over QuickHuskySort, consistent with the 2-4x speedups actually measured (§ 6.3). The model does predict one caveat: equating A_r and A_h gives a theoretical break-even around $N \approx 17$, somewhat higher than Huskysort's own break-even against merge sort ($N = 4$, above) — because radix's per-pass cost has no logarithmic term to shrink relative to at very small N. This crossover is far below every array size actually tested, so it has no practical effect on the results reported here, but it is worth stating rather than leaving the model looking like radix wins unconditionally at every N.

Table 6 was generated by sorting arrays of random English words, sampled with replacement at each size shown from the Leipzig Corpora Collection [Goldhahn et al. 2012], comparing Huskysort, a re-implemented dual-pivot quicksort (see § 5.2), and the Java system sort (Timsort), on the original 2020 environment (Table 2).

Since the final pass of Huskysort will be cleaning up a relatively small number of inversions, does it matter whether we use insertion sort or Timsort? It turns out that for arrays which are smaller than about 50,000 Strings, it makes barely any difference at all. Indeed, insertion sort is sometimes faster. However, as the size of the array increases beyond this value, Timsort begins to win out over insertion sort. Please refer to table 7.

6 RESULTS AND OBSERVATIONS

6.1 Benchmarks

- Comparison of sort times for Huskysort, dual-pivot quicksort, system sort. Please refer to table 6.
- Time vs Size (N) for Huskysort, Please refer to figure 4
- Comparison of Huskysort with system sort for numeric types (Integer, Double, Long, BigInteger, BigDecimal): Table 8 (JMH, $N = 500{,}000$, ms ± 99.9% CI).
- Comparison of Huskysort with system sort for String types (English words, Chinese words): Table 9 (JMH, ms ± 99.9% CI).
- Comparison of Huskysort with system sort for Tuples: Table 10 (JMH, ms ± 99.9% CI).

6.2 Summary

Huskysort's target application is the sorting of randomly ordered arrays of objects on the JVM (Java Virtual Machine). Let's eliminate the non-use-cases first:

- Partially-ordered arrays: that's to say, arrays with only a linear number of inversions as would be the case when adding

Table 5. Theoretical comparisons (array accesses, dimensionless counts)

N	N ln N	merge sort	Huskysort	Radix
4	6	72	72	108
1,000	6,908	81,439	55,075	27,000
1,000,000	13,815,511	160,878,371	101,149,455	27,000,000
1,000,000,000	20,723,265,837	240,317,557,125	147,224,183,132	27,000,000,000

Table 6. Relative sort times for Huskysort, dual-pivot quicksort, system sort (dimensionless ratio, normalized to DP quicksort = 1.00)

N	Huskysort	DP quicksort	system sort
1000	0.60	1.00	0.84
2000	0.78	1.00	1.19
4000	0.81	1.00	1.23
8000	0.77	1.00	1.27
16000	0.74	1.00	1.24
32000	0.74	1.00	1.24
64000	0.85	1.00	1.29
125000	0.76	1.00	1.51
250000	0.92	1.00	1.32
500000	0.97	1.00	1.42
1000000	0.95	1.00	1.45
2000000	0.93	1.00	1.54
4000000	0.91	1.00	1.64

Table 7. Timsort or insertion sort as the cleanup pass (time in milliseconds)

N	Husky with Tim (ms)	Husky with Insertion (ms)
1000	0.14	0.14
2000	0.31	0.30
4000	0.63	0.64
8000	1.46	1.56
16000	3.24	3.19
32000	6.56	6.48
64000	16.85	19.41
125000	47.74	54.90
250000	168.21	167.90
500000	259.37	373.43
1000000	602.08	1202.61
2000000	1234.90	3129.23
4000000	2423.62	11018.19

Table 8. Comparison of Huskysort with system sort (Numeric, $N = 500{,}000$, ms, mean ± 99.9% CI)

Type	System	QuickHuskySort	DualPivotQuicksort	quicksort (raw)	Radix/8	Radix/16
Integer	94.4±4.8	68.9±8.3	67.0±5.8	36.3±3.4	27.3±3.8	26.0±2.8
Double	109.6±15.6	73.5±5.1	94.9±59.4	41.4±1.3	29.1±6.0	22.3±2.1
Long	101.5±8.6	74.9±3.7	77.0±11.3	35.2±0.5	25.4±5.3	38.2±42.1
BigInteger	157.1±10.5	108.6±5.7	157.9±10.3	38.4±2.8	42.4±10.8	35.4±2.1
BigDecimal	201.4±21.2	145.0±136.9	189.6±39.1	46.2±1.9	45.0±8.0	39.4±5.3

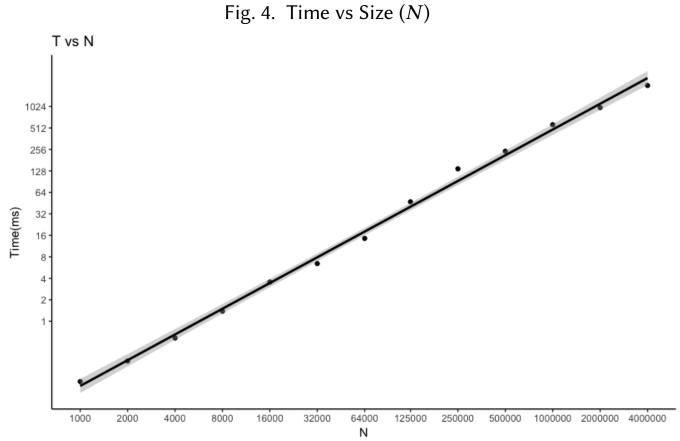
Table 9. Comparison of Huskysort with system sort (Strings, ms, mean ± 99.9% CI)

N	Corpus	System sort	QuickHuskySort	Radix/8	Radix/16
32,000	English	12.25±5.04	12.86±5.05	4.00±0.85	3.79±0.75
32,000	Chinese	11.25±4.24	4.11±0.52	1.82±0.35	1.79±0.81
200,000	English	146.09±111.5	87.18±36.9	50.39±19.6	41.12±4.3
200,000	Chinese	61.65±11.7	25.99±4.0	10.50±1.9	9.76±4.3
1,000,000	English	571.87±53.4	391.09±154.9	260.81±65.8	239.17±49.4
1,000,000	Chinese	287.60±17.4	112.13±7.2	98.26±20.3	85.43±37.2

Table 10. Comparison of Huskysort with system sort (Tuples, ms, mean ± 99.9% CI)

N	System	QuickHuskySort	DualPivotQuicksort	Radix/16
20,000	3.11±0.40	2.19±0.10	2.71±0.13	1.03±0.05
100,000	19.93±1.52	13.63±1.05	19.58±3.50	4.89±0.23
500,000	174.30±13.6	117.14±20.8	155.56±16.1	45.00±4.6

Fig. 4. Time vs Size (N)



a relatively small number of records to a large existing index. For such use cases, Timsort (the Java system sort for objects) would be more suitable.

- Arrays of primitives: in Java, a clear distinction exists between primitive types (char, byte, short, int, float, long, double) and objects. Dual-pivot quicksort (the Java system sort for primitives) is already the fastest algorithm for sorting such types.

Thus, Huskysort is best for any object array where there's no expectation of partial ordering. Here is a summary of typical improvements in performance over Timsort, expressed as a direct multiplier (e.g. **1.5x** means Huskysort took roughly two-thirds the time) rather than a percentage, since a percentage figure is easy to misread: a "42% improvement" and a "1.7x speedup" describe the same measurement, but only one of them is unambiguous about what it's a percentage *of*. Please refer to table 11

6.3 Radix Sort Results

We re-benchmarked every data type and corpus covered above using RadixHuskySort (§ 3.2) in place of Introsort for step 2, using JMH (the Java Microbenchmark Harness, with proper fork isolation and warmup) rather than a hand-timed loop, and on JDK 21 rather than the JDK 1.8.0_152 used for the results above.

Table 12 reports radix's advantage over QuickHuskySort as a direct multiplier — e.g. **2.9x** means radix took roughly a third as long — rather than as a percentage, since a percentage figure close to or above 100% becomes ambiguous about exactly which quantity it is a percentage *of* (a concern raised about the original figures 6-9).

This is worth confronting directly against the original results: Table 6 shows Huskysort's own advantage over dual-pivot quicksort *shrinking* as N grows — from a relative time of 0.60 at $N = 1{,}000$ to 0.97 at $N = 4{,}000{,}000$, i.e. the improvement fades from roughly 40% to roughly 3%, which would undermine confidence that the advantage is durable at larger scale. Radix sort's advantage over QuickHuskySort does not shrink with N in the same way: at every size and data type in Table 12, the margin is 1.3x or greater, consistent with the theoretical model of § 3.2 predicting no logarithmic term at all, so there is no analogous reason to expect the advantage to erode as N grows further. Chinese names sorted by pinyin order are the smallest margin measured (1.6x) because both QuickHuskySort and RadixHuskySort must pay the same collator-based cleanup pass cost for that coding; Dates show the largest margin (4.5x) because that coding is provably perfect and never needs a cleanup pass at all, letting radix's linear-time advantage show through with nothing else in the way.

6.4 Parallel Radix Sort

Radix sort's digit passes are natural candidates for parallelization (§ 3.2 above). We implemented and benchmarked a parallel variant: each pass is split into a configurable number of contiguous chunks, one per worker thread. Every chunk first computes its own local per-bucket histogram independently, with no synchronization required; a short sequential step then combines those histograms

Table 11. Improvements Summary

Data Type	Advantage over Timsort
Strings of English words (4,000–500,000 elements)	1.5-1.7x
Strings of Chinese words (4,000–500,000 elements)	1.4-2.1x
Tuples (20,000-500,000 elements))	1.2x
Integers (20,000-500,000 elements))	1.4-1.7x
Doubles (20,000-500,000 elements))	1.3-1.6x
Longs (20,000-500,000 elements))	1.4-1.7x
BigIntegers (20,000-500,000 elements))	1.5-1.7x
BigDecimals (20,000-500,000 elements)	1.6x

Table 12. Radix sort advantage over QuickHuskySort

Data Type	N	Advantage over QuickHuskySort
Strings of English words	1,000,000	1.6x
Strings of Chinese words (natural order)	1,000,000	1.3x
Strings of Chinese names (pinyin order)	1,000,000	1.6x
Integer	500,000	2.9x
Double	500,000	3.3x
Long	500,000	2.9x
BigInteger	500,000	3.1x
BigDecimal	500,000	3.7x
Tuples	500,000	2.6x
Dates	20,000	4.5x

into an exact starting offset, in the output array, for every (chunk, bucket) pair — preserving the stability that step 3's cleanup pass, when needed, relies on, since a chunk's elements always land after all lower-numbered chunks' elements sharing the same bucket; finally, every chunk scatters its own elements into the output arrays independently, using only its own precomputed offsets, so no two chunks ever write to the same location.

An initial implementation dispatched the two phases above to a thread pool separately on every pass. Measurement showed this repeated dispatch itself contributing a real, non-trivial share of the total cost, particularly at more moderate array sizes. The overhead was substantially reduced by starting the worker threads once per sort, rather than once per phase per pass: each thread runs its own loop over every pass, and the two phases synchronize via reused barriers whose associated actions — code that runs exactly once per synchronization point, in whichever thread arrives last — perform the histogram-combine and buffer-swap steps. Table 13 reports the result (*Long[]*, 11-bit digits, milliseconds, mean ± 99.9% CI).

Isolating parallelism itself — comparing 1 thread against 4, both paying identical thread-coordination overhead — gives a real, statistically resolved speedup at both sizes: 1.41x at $N = 2{,}000{,}000$ and 1.61x at $N = 10{,}000{,}000$, non-overlapping confidence intervals in both cases. Against the existing serial implementation, the improvement is resolved at $N = 10{,}000{,}000$ (574.3ms to 426.4ms, 1.35x) and directionally consistent but not fully resolved at $N = 2{,}000{,}000$ (1.32x, overlapping intervals). Four threads is the practical sweet spot on the machine used for this measurement (four performance and four efficiency cores): eight threads performs statistically indistinguishably from four at both sizes, consistent with only four of the eight cores being full-performance ones.

7 CONCLUSION

We have demonstrated that we can shift work from the linearithmic phase of sorting to the linear phase (both the prelude and the postlude) and effect an overall reduction in array accesses and thus processing time. When sorting objects rather than primitives, Huskysort is always faster than dual-pivot quicksort. It is faster than Timsort for arrays which are not partially ordered. It is especially fast for Unicode character strings.

ACKNOWLEDGMENTS

To Darshan Dedhia, Akshay Bhusare, Kartik Kumar for their help with this project.

REFERENCES

Alok Aggarwal and Jeffrey Scott Vitter. 1988. The Input/Output Complexity of Sorting and Related Problems. *Commun. ACM* 31, 9 (Sept. 1988), 1116–1127. https://doi.org/10.1145/48529.48535

Nicolas Auger, Vincent Jugé, Cyril Nicaud, and Carine Pivoteau. 2018. On the Worst-Case Complexity of TimSort. *CoRR* abs/1805.08612 (2018). arXiv:1805.08612 http://arxiv.org/abs/1805.08612

Jon L. Bentley and M. Douglas McIlroy. 1993. Engineering a sort function. *Software: Practice and Experience* 23, 11 (1993), 1249–1265. https://doi.org/10.1002/spe.4380231105

Jon L. Bentley and Robert Sedgewick. 1997. Fast Algorithms for Sorting and Searching Strings. In *Proceedings of the Eighth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '97)*. Society for Industrial and Applied Mathematics, USA, 360–369.

Table 13. Parallel radix sort: serial vs. 1, 2, 4, and 8 worker threads

N	Serial	1 thread	2 threads	4 threads	8 threads
2,000,000	100.4±24.0	107.5±10.7	83.7±3.3	76.3±4.1	77.3±3.6
10,000,000	574.3±83.3	685.3±135.5	569.5±307.0	426.4±49.1	444.0±127.3

M. Frigo, C. E. Leiserson, H. Prokop, and S. Ramachandran. 1999. Cache-oblivious algorithms. In *40th Annual Symposium on Foundations of Computer Science (Cat. No.99CB37039)*. 285–297. https://doi.org/10.1109/SFFCS.1999.814600

William F. Gilreath. 2004. Hash sort: A linear time complexity multiple-dimensional sort algorithm. *CoRR* cs.DS/0408040 (2004). http://arxiv.org/abs/cs.DS/0408040

Dirk Goldhahn, Thomas Eckart, and Uwe Quasthoff. 2012. Building Large Monolingual Dictionaries at the Leipzig Corpora Collection: From 100 to 200 Languages. In *Proceedings of the Eight International Conference on Language Resources and Evaluation (LREC'12)* (23-25), Nicoletta Calzolari (Conference Chair), Khalid Choukri, Thierry Declerck, Mehmet Uğur Doğan, Bente Maegaard, Joseph Mariani, Asuncion Moreno, Jan Odijk, and Stelios Piperidis (Eds.). European Language Resources Association (ELRA), Istanbul, Turkey.

C. A. R. Hoare. 1961. Algorithm 64: Quicksort. *Commun. ACM* 4, 7 (July 1961), 321. https://doi.org/10.1145/366622.366644

Vincent Jugé. [n.d.]. *Adaptive Shivers Sort: An Alternative Sorting Algorithm.* 1639–1654. https://doi.org/10.1137/1.9781611975994.101 arXiv:https://epubs.siam.org/doi/pdf/10.1137/1.9781611975994.101

Anthony LaMarca and Richard E. Ladner. 1999. The Influence of Caches on the Performance of Sorting. *Journal of Algorithms* 31, 1 (1999), 66–104. https://www.sciencedirect.com/science/article/pii/S0196677498909853

Peter McIlroy. 1993. Optimistic Sorting and Information Theoretic Complexity. In *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms* (Austin, Texas, USA) *(SODA '93)*. Society for Industrial and Applied Mathematics, USA, 467–474.

DAVID R. MUSSER. 1997. Introspective Sorting and Selection Algorithms. *Software: Practice and Experience* 27, 8 (1997), 983–993. https://doi.org/10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-# arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/

T Peters. 2002. *Timsort - Python.* Retrieved November 3, 2020 from https://svn.python.org/projects/python/trunk/Objects/listsort.txt

Robert Sedgewick. 1996. Analysis of Shellsort and related algorithms. In *Algorithms — ESA '96*, Josep Diaz and Maria Serna (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 1–11.

Ranjan Sinha and Justin Zobel. 2004. Cache-Conscious Sorting of Large Sets of Strings with Dynamic Tries. *ACM J. Exp. Algorithmics* 9 (2004), 1.5. https://doi.org/10.1145/1064546.1180622

W. Xiang. 2011. Analysis of the Time Complexity of Quick Sort Algorithm. In *2011 International Conference on Information Management, Innovation Management and Industrial Engineering*, Vol. 1. 408–410. https://doi.org/10.1109/ICIII.2011.104

Vladimir Yaroslavskiy. 2009. *Dual Pivot Quicksort.* Retrieved November 3, 2020 from https://codeblab.com/wp-content/uploads/2009/09/DualPivotQuicksort.pdf

Hantao Zhang, Baoluo Meng, and Yiwen Liang. 2016. Sort Race. arXiv:1609.04471

A APPENDIX

A.1 Discussion of coding accuracy

It is not essential that coding is perfect for Huskysort to work well. We have experimented with deliberately throwing off the encoding quite drastically. If the probability of one of these deliberate mis-codings is less than approximately 25 percent, Huskysort can still save time. The inference that we can take from this is that it is not critical that encodings are all very good. We can be confident that we are still saving time up to about one in four errors.

These deliberate errors were tested for Bytes and Integers but not the other generic types.

A.2 Adversarial inputs

The discussion in section A.1 raises a concern: it would be entirely possible to construct inputs on which the husky encoding has poor entropy, and it is not obvious in advance how gracefully (or badly) the sort that follows would degrade in that case. We constructed two such inputs deliberately, to see how the two variants (QuickHuskySort and RadixHuskySort, § 3.2) each degrade.

Collapsed high-order bits. We generated arrays of *Long* values with a swept number of high-order bits held identical across every element (the rest random) — 0 fixed bits is the random baseline, 63 leaves only the sign bit free — and compared against a from-scratch re-implementation of dual-pivot quicksort (the same baseline used for the numeric comparisons above, without any three-way/equal-key handling). Table 14 reports timings (milliseconds) at $N = 1{,}000{,}000$.

Radix sort's timings stay essentially flat across the entire sweep, exactly as the model of § 3.2 predicts: each digit pass does the same fixed amount of work regardless of how the keys are distributed, so a pass over collapsed digits (everything lands in one bucket) costs no more than any other pass. The from-scratch quicksort baseline degrades by more than an order of magnitude and then fails outright with a stack overflow — a concrete instance of exactly the kind of adversarial input this concern anticipates, though notably it is the *baseline* implementation that fails, not QuickHuskySort itself. This is a well-known failure mode for a quicksort implementation that partitions naively around a heavily-duplicated pivot value: Bentley and McIlroy's classic treatment of engineering a practical quicksort [Bentley and McIlroy 1993] specifically addresses this case (a solution to Dijkstra's Dutch National Flag problem, partitioning into three regions — less than, equal to, and greater than the pivot — rather than two), precisely to avoid the pathological recursion depth this baseline exhibits. QuickHuskySort (with its depth-limited heapsort fallback) neither crashes nor slows down anywhere in this sweep — if anything it gets faster as duplication increases, the same pattern System sort shows, consistent with Timsort's adaptive run-detection treating long runs of near-equal keys as already sorted. Radix sort is more robust still, because its immunity to this pattern is structural rather than incidental to a fallback mechanism.

Shared string prefixes. We took real corpus words and prepended a fixed-length prefix of repeated characters, long enough in the worst case to exceed the English coder's character-capture window (nine 7-bit ASCII characters per 64-bit code). Table 15 reports timings at $N = 1{,}000{,}000$.

This is a genuinely different failure mode, and a less flattering one. Once the shared prefix reaches the coder's nine-character window, every element's husky code becomes bit-for-bit identical — the encoding provides zero disambiguation — and *every* Husky-based approach, radix included, ends up slower than plain System sort, not merely no-longer-faster. The mechanism has nothing to do with which algorithm sorts the codes in step 2: once the encoding carries no information at all, the entire ordering burden falls on the mandatory cleanup pass (step 3) doing real string comparisons —

Table 14. Timings (ms) as high-order bits are progressively collapsed, N = 1,000,000

Fixed high bits	System sort	QuickHuskySort	Raw quicksort	Radix/16
0	287.9	180.6	174.9	61.5
32	232.0	193.4	180.2	53.5
48	205.5	159.5	163.4	51.3
56	107.4	82.6	3220.5	52.2
60	71.5	75.6	*crashed*	40.9
63	27.1	13.1	*crashed*	34.5

Table 15. Timings (ms) as a shared string prefix grows, N = 1,000,000

Prefix length	System sort	QuickHuskySort	Radix/16
0	627.0	292.1	180.2
10	452.9	484.9	519.9
20	439.7	475.7	568.1
40	516.7	556.7	582.3

comparisons that are themselves more expensive than usual, since they must scan past the shared prefix before finding a difference — and whichever algorithm ran first in step 2 did genuinely useless work before that. Radix does not recover any advantage here, and is very slightly the slowest of the three Husky-based options in this regime, since its fixed per-pass cost (a strength above) becomes pure overhead once there is no information left to sort.

Taken together, these two experiments answer the question posed above with two distinct results. When keys collide in their high-order bits but the encoding itself is still informative, radix sort is effectively immune, and a naively-implemented quicksort baseline is not — it can degrade by orders of magnitude or fail outright. But when the source data overwhelms the encoding's fixed capture window entirely, no choice of sort algorithm for the encoded longs helps at all; this is a limitation of the fixed-width encoding scheme itself, not of Huskysort's algorithmic strategy, and it is already implicit in the p_{crit} discussion of § 3.4: whenever the actual input defeats the encoding, the honest answer is that no downstream sort algorithm can buy back the information the encoding failed to capture.

A.3 Handling of partially-sorted arrays

Not surprisingly, Huskysort is best suited for sorting random arrays. The benefits of partially-sorted arrays are enjoyed most by the merge sort family of algorithms (including Timsort). However, we have studied the efficacy of a modified merge sort (it is required to manage both the objects and the longs) as the first pass of the Huskysort algorithm. For random English String inputs, the merge-sort variation improves on system sort by approximately 20 percent whereas the normal intro-sort-based Huskysort improves on the system sort by more like 40 Percent.

However, it is not recommended to use Huskysort when arrays are partially ordered. In such cases, the merge-sort variation of Huskysort will take anywhere between one and one half times and four times as long as Timsort.